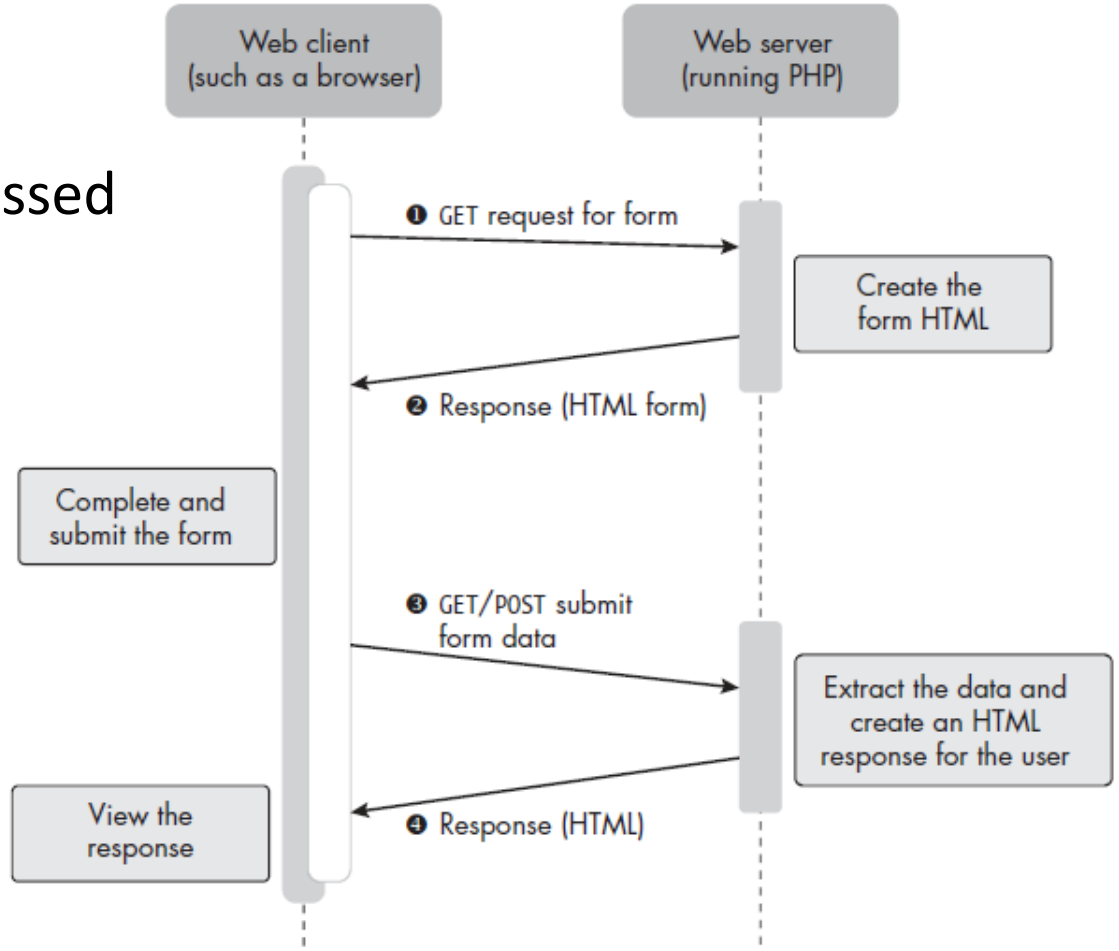
CSIS 3280: Lecture 4

Processing Web Forms (CH11)

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Client/Server Communication for Web Forms

- Behind the typical web form lies a sequence of four messages between a web client (such as the user's browser) and a web server, whereby the form is requested, received, submitted, and processed



© Copyright Bambang and others. NOT FOR DISTRIBUTION

GET vs POST

- The data variables sent from web forms to the server are simple name/value pairs, such as `username=matt` or `q=chocolate`
- GET method
  - An HTTP GET request is primarily for retrieving data or a web page from a server.
    - GET request should never result in changes to content stored on the server, e.g., database
  - One common use of the GET method is to create a URL that's easily bookmarked or shared
  - Special characters and spaces can't be represented verbatim in a query string but must instead be encoded as other symbols. For example, each space is replaced with either %20 or a plus sign +
  - A GET request is created by either the result of query-string variables come from form submissions or were hardcoded. The name/value pairs can be extracted by a server-side script for processing

© Copyright Bambang and others. NOT FOR DISTRIBUTION

GET vs POST

- POST method
  - With the POST method, you can send variables in the body of the HTTP message, meaning they won't be seen in the resulting URL.
    - Useful to For any confidential data like usernames and passwords
  - These values therefore don't appear as part of the URL in the address bar.
  - POST request can send data directly in the query string, like a GET request, as well as in the request body

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Processing GET Request Data

- Recall the following code from Lecture 1

```php
<?php
    $secretNumber = 777;
    if ($_GET['guess'] == $secretNumber) {
        echo "<p>You're a wizzard!!</p>";
    } else {
        echo "<p>Try again next time!</p>";
    }
?>
```

The $_GET and $_POST arrays are examples of PHP's superglobals. These are arrays that always exist and that can be accessed from anywhere in your PHP code (that is, from any scope), including inside functions and class methods.

- Assuming that the code was stored at the Webroot and accessed with the following code (without any query string), an error message will be generated http://localhost/if_else.php
- We need to check whether the array key 'guess' exists:

```php
<?php
    $secretNumber = 777;
    if (isset($_GET['guess']) && ($_GET['guess'] == $secretNumber)) {
        echo "<p>You're a wizzard!!</p>";
    } else {
        echo "<p>Try again next time!</p>";
    }
?>
```

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Processing GET Request Data

- Consider the following code snippets

```
<!doctype html><html><head><title>simple form</title></head>
<body>
<form method="GET" action="process.php">
    <input name="firstName">
    <input type="submit">
</form>
</body>
</html>
```

index.php

```
<?php
//----- (1) LOGIC -----
$firstName = filter_input(INPUT_GET, 'firstName');
?>

<!-- (2) HTML template output -->
<!doctype html> <html><head><title>process</title></head><body>
Hello <?= $firstName ?>
</body></html>
```

process.php

The INPUT_GET argument to filter_input() is a bit misleading. Its purpose is to retrieve data from the URL query string, regardless of whether that data was sent via the GET or POST method, since POST method can also send the form data to either message body and URL query string

However, by using filter_input() we do not need to check whether the key of either $_GET or $_POST exists since it has its own built-in isset() check

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Processing POST Request Data

- Consider the following code snippets

```
<!doctype html><html><head><title>simple form</title></head>
<body>
<form method="POST" action="process.php">
    <input name="firstName">
    <input type="submit">
</form>
</body>
</html>
```

index.php

```
<?php
//----- (1) LOGIC -----
$firstName = filter_input(INPUT_POST, 'firstName');
?>

<!-- (2) HTML template output -->
<!doctype html> <html><head><title>process</title></head><body>
Hello <?= $firstName ?>
</body></html>
```

process.php

When working with POST method, we should change the web form's method into POST and change the constant in filter_input() to be INPUT_POST

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Processing Mixed GET & POST

- Consider the following code snippets

```html
<!doctype html><html><head><title>Movie Form 1</title></head><body>
    <h1>Edit movie</h1>
    <form method="POST" action="/01_mixed_request_process.php?id=1022">
        <label for="title">Title: </label><input name="title" id="title">
        <br>
        <label for="price">Price: </label><input name="price" id="price">
        <br>
        <input type="submit">
    </form>
    </body>
</html>
```

01_mixed_request.html

```php
<?php
//----- (1) LOGIC -----
$id = filter_input(INPUT_GET, 'id');
$title = filter_input(INPUT_POST, 'title');
$price = filter_input(INPUT_POST, 'price');
?>

<!-- (2) HTML template output -->
<!doctype html> <html><head><title>process</title></head><body>
id = <?= $id ?>
<br>title = <?= $title ?>
<br>price = <?= $price ?>
</body></html>
```

01_mixed_request_process.php

The form's action includes a query string id=1022
- A GET request
- A 'non-editable' form data

You can achieve a better result by using a hidden HTML input, hence you only need to work with INPUT_POST

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Multiple Submit Buttons

- In some use cases, you may want to provide multiple submit button
  - The name attributes of those buttons should not be similar

```
<form method="POST" action="02_multiple_process.php?movieId=80441">
    <p><label for="number">Credit card number:</label>
        <input name="number" id="number"></p>
    <p><label for="date">
            Expiry date:</label>
        <input name="date" id="date"></p>
    <p><label for="ccv">
            CCV code:</label>
        <input name="ccv" id="ccv"></p>
    <p>
        <input type="submit" name="watchNow"
            value="Pay and start watching now" class="btn btn-success">
        <input type="submit" name="watchLater"
            value="Pay and watch later" class="btn btn-success">
    </p>
</form>
```

02_multiple_buttons.html

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Multiple Submit Buttons

- Processing multiple submit button
  - You need to check both name/value pairs using filter_has_var()

```php
<?php
if (filter_has_var(INPUT_POST, 'watchNow')) {
    print 'you clicked the button to <b>Watch Now</b>';
} else {
    print 'you clicked the button to <b>Watch Later</b>';
}
```

02_multiple_process.php





© Copyright Bambang and others. NOT FOR DISTRIBUTION

Creating Links with Query String

- You can use the <a> tag of HTML to create button containing links with query string. However, it is better to implement them with your PHP code

03_button_links.php

```php
<?php
// setup data array
$items = [
    ['id' => 102, 'description' => 'Hammer', 'price' => 9.99],
    ['id' => 511, 'description' => 'Bucket of nails', 'price' => 19.99],
];
?>
<!doctype html><html><head><title>Basket From Array</title>
    <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    >
</head>
<body class="container">
<h1>Your shopping basket</h1>
<?php foreach ($items as $item): ?>
    <div class="col-lg-3 text-end py-2">
        <?= $item['description'] ?> $ <?= $item['price'] ?>
        <a href="/show.php?id=<?= $item['id'] ?>" class="btn btn-primary">Details</a>
    </div>
<?php endforeach; ?>
</body>
</html>
```

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Working with Radio Buttons

- Radio buttons are mutually exclusive since user can select only one radio
  - The name attribute for both radio should be similar
  - It can be processed using name/value pair extraction easily

```
<!doctype html><html><head><title> Radio Buttons </title>
    <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    >
</head>
<body class="container">
<form method="GET" action="process.php">
    <br>
    <label>
        <input type="radio" name="county" value="dublin" checked>
        Dublin
    </label>

    <br>
    <label>
        <input type="radio" name="county" value="cork">
        Cork
    </label>
    <br>
    <input type="submit" class="btn btn-primary">
</form>
</body>
</html>
```

Working with Checkboxes

- Checkboxes are not mutually exclusive
- You can ***treat each checkboxes individually***
  - The name attributes for these checkboxes MUST NOT similar
  - It can be useful when you have a small number of checkboxes and/or you want to process simple combinations or existence of any of the checkboxes' value

```
<form method="GET" action="process.php">
    <p><label><input type="checkbox" name="olives"> Olives</label></p>
    <p><label><input type="checkbox" name="pepper"> Pepper</label></p>
    <p><label><input type="checkbox" name="garlic"> Garlic salt</label></p>
    <p><input type="submit" class="btn btn-primary"></p>
</form>
```

```
<?php
$olivesSelected = filter_has_var(INPUT_GET, 'olives');
```

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Working with Checkboxes: Using Array

- It is more practical to ***treat checkboxes submitted values as an array***
  - The name attribute' of the checkboxes need to be similar but indicating that it is an array by adding a square bracket []

```
<form method="GET" action="04_checkboxes_process.php">
    <p><label>
        <input type="checkbox" name="toppings[]" value="olives"> Olives
    </label></p>
    <p><label>
        <input type="checkbox" name="toppings[]" value="pepper"> Pepper
    </label></p>
    <p><label>
        <input type="checkbox" name="toppings[]" value="garlic"> Garlic salt
    </label></p>
    <p><input type="submit" class="btn btn-primary"></p>
</form>
```

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Working with Checkboxes: Using Array

- The processing script's filter_input requires us to specify FILTER_REQUIRE_ARRAY constant argument
  - filter_input third parameter is related to ***filter***. If we want to use the default value, we can skip sending the parameter
  - In addition, since the ***options*** is fourth parameter, we need to send a named argument

```php
<?php
$toppings
    = filter_input(INPUT_GET, 'toppings', options: FILTER_REQUIRE_ARRAY);

if (empty($toppings)) {
    print 'no extra toppings selected';
} else {
    $toppingsString = implode('+', $toppings);
    print "toppings: $toppingsString";
}
```

04_checkboxes_process.php

- You can loop through the checkboxes array using any loop structure. You can also use implode to create a string from the array

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Working with Selection List

- With a selection list, the chosen option will be the name/value pair sent to the URL request
  - A simple filter_input with INPUT_GET or INPUT_POST will be enough to process the submitted data

```
<!doctype html><html><head><title>Single Selection list</title>
    <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    >
</head>
<body class="container">
<h1>Flower selection</h1>
<form method="GET" action="process.php">
    <div class="row">
        <div class="col-lg-3 text-end py-2">
            <input type="submit" value="Select a flower" class="btn btn-primary">
        </div>
        <div class="col-lg-3 text-end py-2">
            <select name="flower" class="custom-select">
                <option value="poppy">Poppy</option>
                <option value="daisy">Daisy</option>
                <option value="tulip">Tulip</option>
            </select>
        </div>
    </div>
</form>
</body>
</html>
```

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Working with Multiple Selection List

- With a multiple selection list, you need to treat the options as an array
  - The name attribute' of the select should indicate that it is an array by adding a square bracket []

```
<form method="GET" action="05_multiple_process.html">
    <div class="row">
        <div class="col-lg-3 text-end py-2">
            <input type="submit" value="Select a flower" class="btn btn-primary">
        </div>
        <div class="col-lg-3 text-end py-2">
            <select name="flowers[]"
                    class="custom-select"
                    size="3"
                    multiple>
                <option value="poppy">Poppy</option>
                <option value="daisy">Daisy</option>
                <option value="tulip">Tulip</option>
            </select>
        </div>
    </div>
</form>
```

05_multiple_select.php

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Working with Multiple Selection List

- Similar with checkboxes that was treated as an array, the processing script's filter_input requires us to specify FILTER_REQUIRE_ARRAY constant argument
  - filter_input third parameter is related to ***filter***. If we want to use the default value, we can skip sending the parameter
  - In addition, since the ***options*** is fourth parameter, we need to send a named argument

```php
<?php
$flowers = filter_input(INPUT_GET, 'flowers', options: FILTER_REQUIRE_ARRAY);

if (empty($flowers)) {
    print 'no flowers selected';
} else {
    $toppingsString = implode(' + ', $flowers);
    print "flowers: $toppingsString";
}
```

05_multiple_process.php

© Copyright Bambang and others. NOT FOR DISTRIBUTION

FORM VALIDATION

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Form Validation

- Not all data received through web forms will be valid:
  - users may make mistakes or miss required values, or a number of other things could go wrong.
- HTML forms submit values only as text strings, regardless of the input type
  - It is easy to treat a string containing digits as a number without explicitly type-casting or including data-type conversion
- It is important to carefully write (and test) the validation rules for processing received form data

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Validation process

- Check whether the form was submitted
- **Validate:**
  - **Any required data that's missing** You can test for this with the empty() function.
  - **Too few characters for text data** You might need to ensure that the input meets minimum-length rules for usernames or passwords, for example. You can test for this with the strlen() function.
  - **Non-numeric values** You might want to know if the value received is neither a valid integer nor a decimal number. You can test for this with the is_numeric() function.
- Proceed with a feedback to the user about the data, e.g., display some message, proceed with database access, etc.

© Copyright Bambang and others. NOT FOR DISTRIBUTION

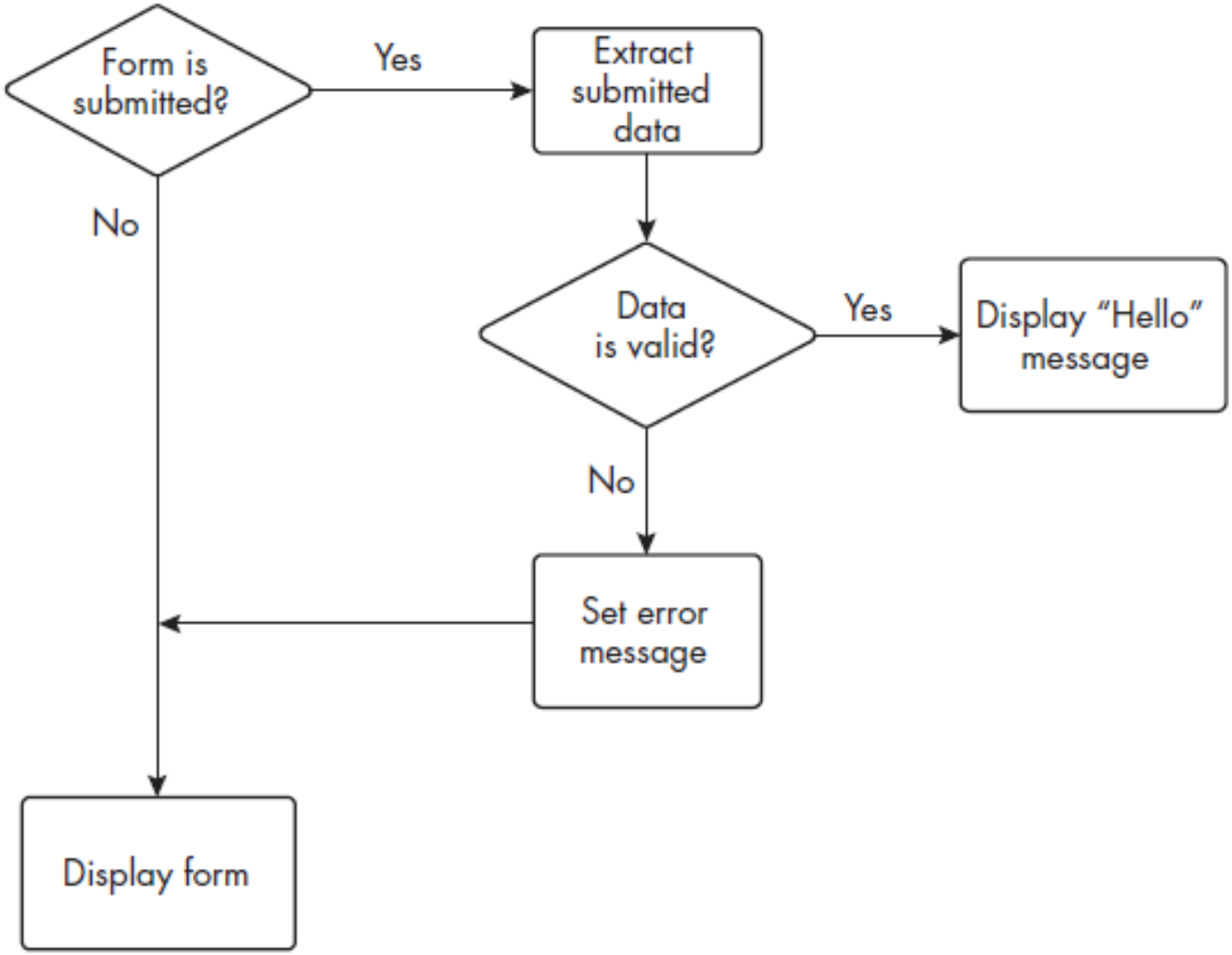
Testing for a Valid Zero Value

- In PHP the following (among other) are considered false:
  - The Boolean false itself
  - The integers 0 and -0
  - The floats 0.0 and -0.0
  - The empty string and the string "0"
  - The special NULL type (including unset variables)
- You can use the identity operator **===**; as well as **gettype()**, **empty()** or **isset()** functions for some confusing values; or **intval()** if you want to check for numerical value

Function call	Return value
`intval('0')`	0
`intval('00')`	0
`intval('0.00')`	0
`intval('0.99')`	0 (decimal component ignored)
`intval('0005')`	5 (leading zeros ignored)
`intval('5abc')`	5 (everything from 'a' on ignored)

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Simple Validation



© Copyright Bambang and others. NOT FOR DISTRIBUTION

Simple Validation

```php
<?php
$isSubmitted = ($_SERVER['REQUEST_METHOD'] === 'POST');
$isValid = true;
$firstName = '';

if ($isSubmitted) {
    $firstName = filter_input(INPUT_POST, 'firstName');

    if (strlen($firstName) < 3) {
        $isValid = false;
        $errorMessage = 'invalid - name must contain at least 3 letters';
    }
}

if ($isSubmitted && $isValid) {
    print "Hello $firstName";
    die(); // End script processing here
}
?>

<!doctype html>
<html><head>
    <title>Tiny sticky postback form</title>
    <style>.error { background-color: pink; padding: 1rem;}</style>
</head>
<body>
<form method="POST">
    <?php if ($isSubmitted && !$isValid): ?>
        <div class="error"><?= $errorMessage ?></div>
    <?php endif; ?>

    <input name="firstName" value="<?= $firstName ?>">
    <input type="submit">
</form>
</body></html>
```

Check whether the form was submitted or not

Initiate isValid flag variable

Set a default value

A simple validation

Confirmation of valid submission

Display error message

Notice that the logic processing in PHP needs to be completed first before you can display back the form with error message

Array Based Error Validation Logic

- The logic is similar with the simple validation; however, the error messages were stored in an array

```php
if ($isSubmitted) {
    $productCode = filter_input(INPUT_POST, 'productCode');
    $price = filter_input(INPUT_POST, 'price');

    if (empty($productCode)) {
        $errors[] = 'missing product code';
    } elseif (strlen($productCode) < 3) {
        $errors[] = 'product code too few characters';
    }

    if (isAnEmptyNonZeroString($price)) {
        $errors[] = 'missing price';
    } elseif (!is_numeric($price)) {
        $errors[] = 'price was not a number';
    }
}
```

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Array Based …: Modular implementation

- We can create several include files containing functions to separate the logic
  - An include file for the HTML template as display, i.e., the view
  - An include file for the validation logic processing, i.e., the model
  - The main file as the routing controller

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Array Based …: Modular implementation

```php
<?php
function displayForm($isSubmitted='', $isValid=true, $errors=[], $productCode='' ,$price=''){
?>
    <!doctype html>
    <html><head>
        <title>Two-field postback form</title>
        <style>.error { background-color: pink; padding: 1rem;}</style>
    </head>
    <body>
    <?php if ($isSubmitted && !$isValid): ?>
        <div class="error">
            <ul>
                <?php foreach ($errors as $error): ?>
                    <li><?= $error ?></li>
                <?php endforeach; ?>
            </ul>
        </div>
    <?php endif; ?>

    <h1>Product details</h1>
    <form method="POST">
        Product code: <input name="productCode" value="<?= $productCode ?>">
        <p>Price: <input name="price" value="<?= $price ?>"></p>
        <p><input type="submit"></p>
    </form>
    </body></html>
<?php
}
```

inc/form.php

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Array Based …: Modular implementation

```php
<?php
function validate(){
    $errors = [];

    $productCode = filter_input(INPUT_POST, 'productCode');
    $price = filter_input(INPUT_POST, 'price');

    if (empty($productCode)) {
        $errors[] = 'missing product code';
    } elseif (strlen($productCode) < 3) {
        $errors[] = 'product code too few characters';
    }

    if (isAnEmptyNonZeroString($price)) {
        $errors[] = 'missing price';
    } elseif (!is_numeric($price)) {
        $errors[] = 'price was not a number';
    }

    return $errors;
}
```

inc/validate.php

© Copyright Bambang and others. NOT FOR DISTRIBUTION

Array Based …: Modular implementation

06_validate_error_modular.php

```php
<?php

require_once 'inc/zeroFunction.php';
require_once 'inc/validate.php';
require_once 'inc/form.php';

$isSubmitted = ($_SERVER['REQUEST_METHOD'] === 'POST');
$productCode = '';
$price = '';
$errors = [];

if($isSubmitted){
    $errors = validate();
}

$isValid = empty($errors);

if ($isSubmitted && $isValid) {
    print 'input data was error free';
    die(); // End script processing here
}
else{
    displayForm($isSubmitted,$isValid,$errors,$productCode,$price);
}
```

© Copyright Bambang and others. NOT FOR DISTRIBUTION